

A Matrix Multiplication Accelerator Based on Systolic Array and Tiling

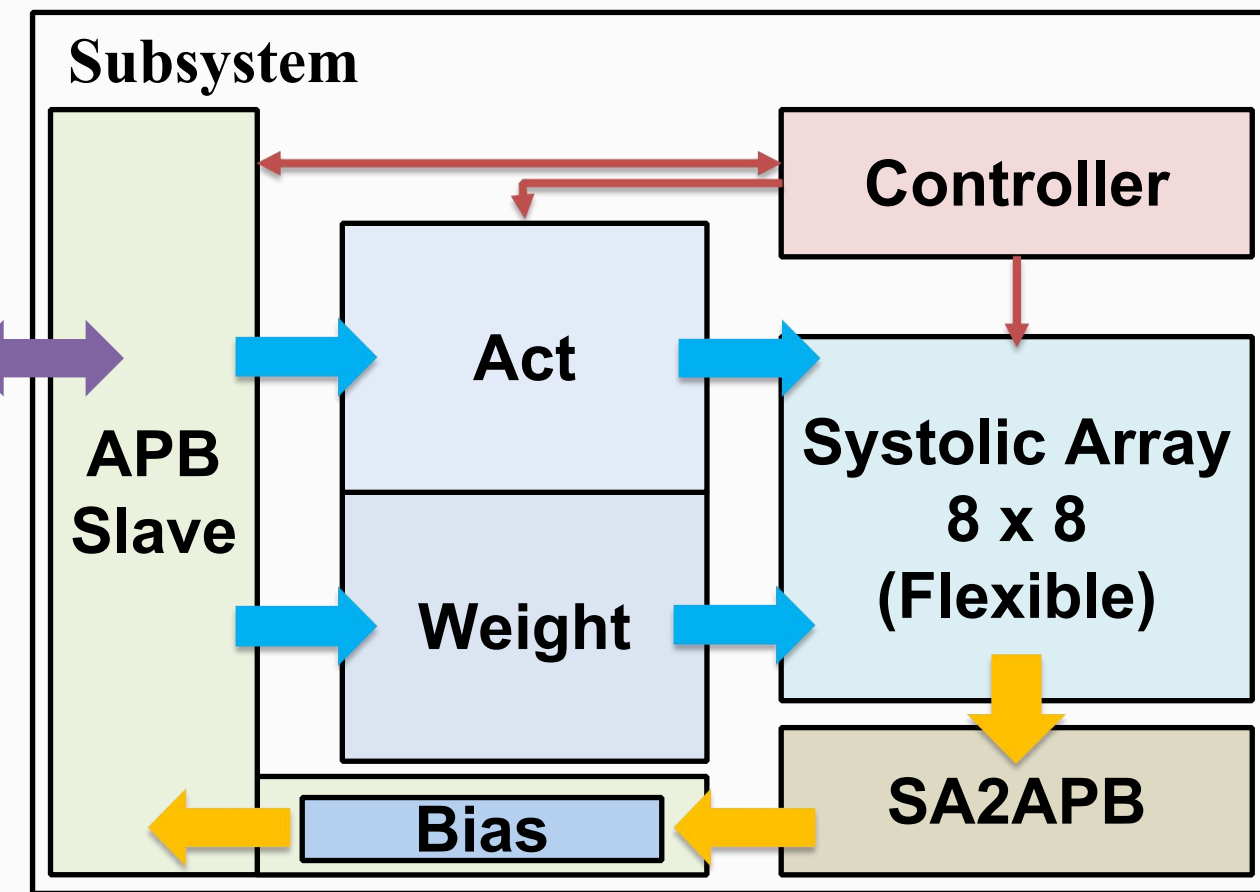


Zihan Zhang · Mi Zhou

Supervisors: Paul R. Genssler · Wu-Young Jang

Introduction

While traditional matrix multiplication is highly time-consuming, an acceleration system based on a systolic array and tiling strategy is introduced. The original matrices are stored in a global SRAM, and an 8×8 systolic array tile is reused to compute the whole matrices. The subsystem is integrated into the Didactic_SoC platform via an APB bus.

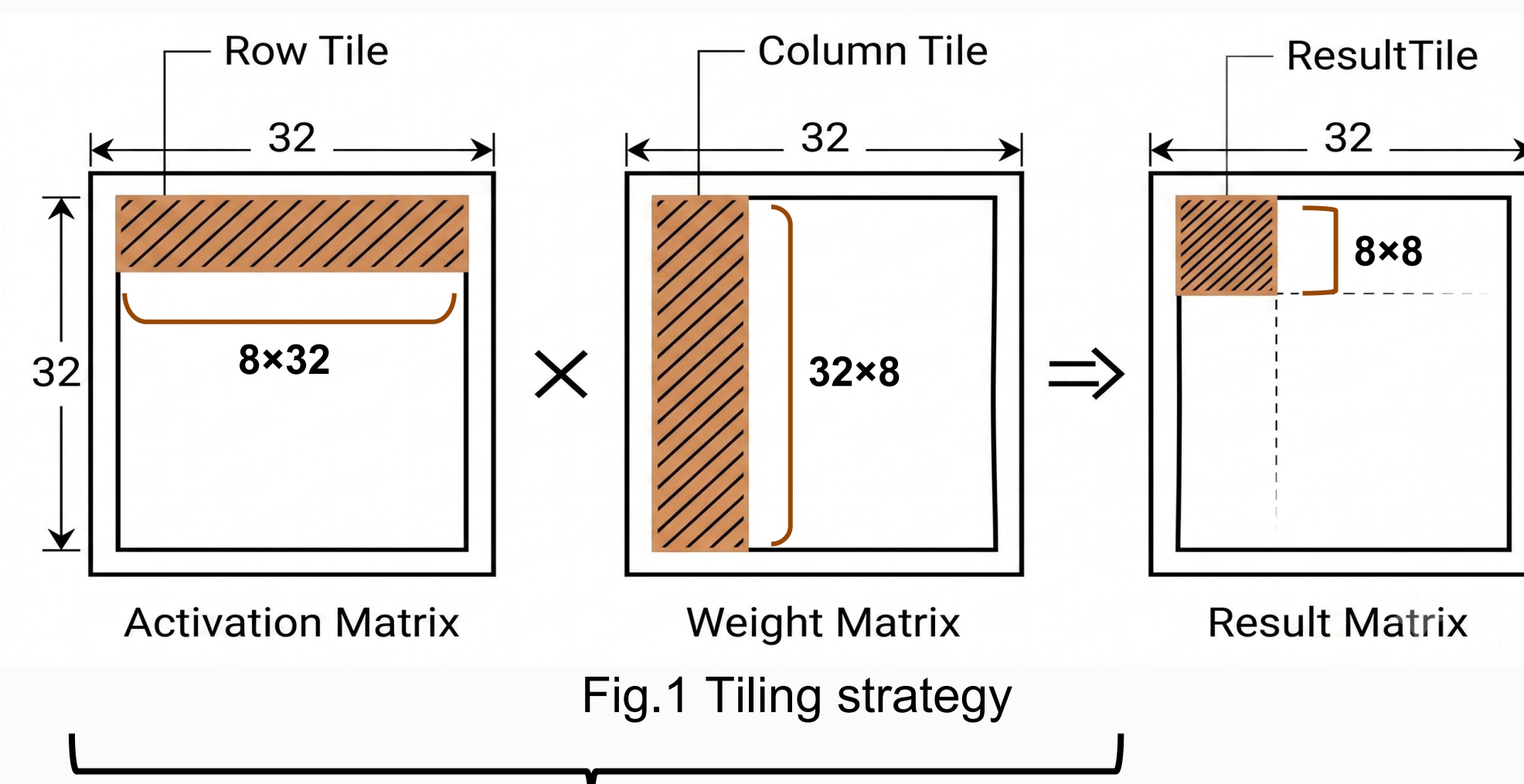
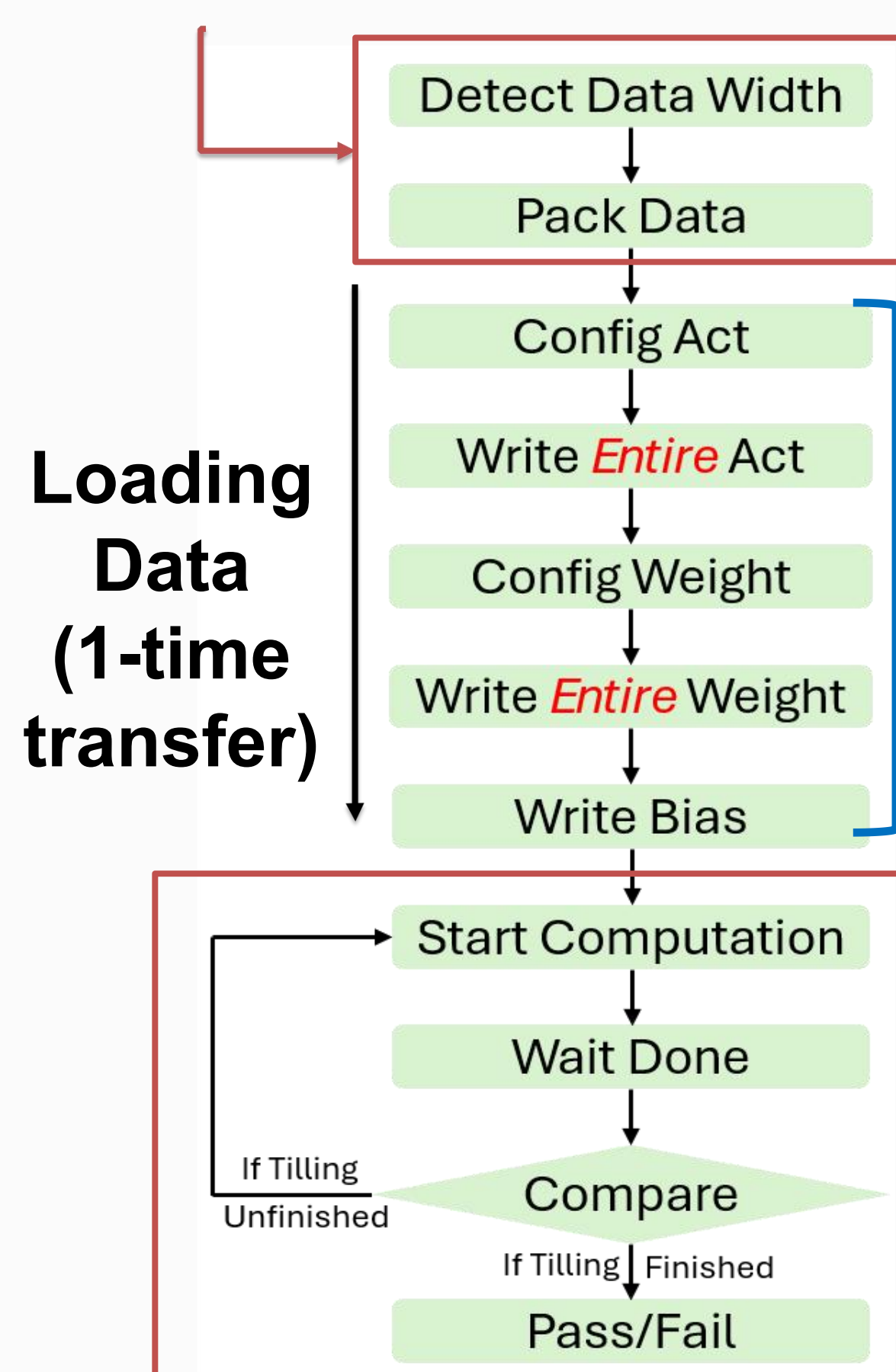


Highlights

- **Systolic Array:** Enables staggered parallel data flow into the Mac array and all units compute synchronously. The controller enables rows and columns' MAC units precisely.
- **Tiling:** Processes data iteratively in a small tile, dynamically adapting to hardware resources. Data is stored in a global SRAM, avoiding repeated low-speed APB transfers.
- **Adaptive MAC Unit:** Supports multi-precision signed integer computation from 8-bit to 32-bit. The internal multiplier can be configured as a combinational or a multi-cycle Booth multiplier.

Implementation Details

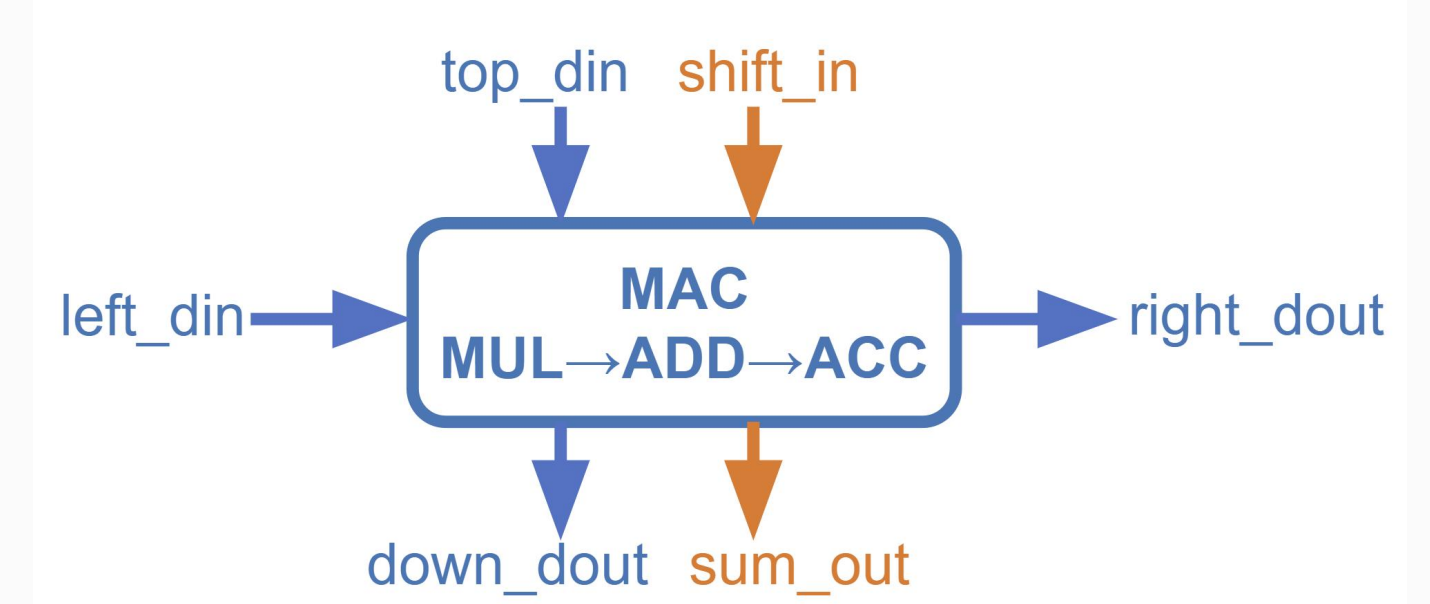
Before Computation: the Software Driver pre-processes the input data and configures the accelerator according to the matrix data format. The data width of the activation and weight matrix is automatically detected, **supporting multiple precision modes**. Since the activation and weight matrices may use different data widths, the driver configures them separately. Also, to make the mix-precision data transfer more efficient, low-precision matrix elements are packed into 32bit before written.



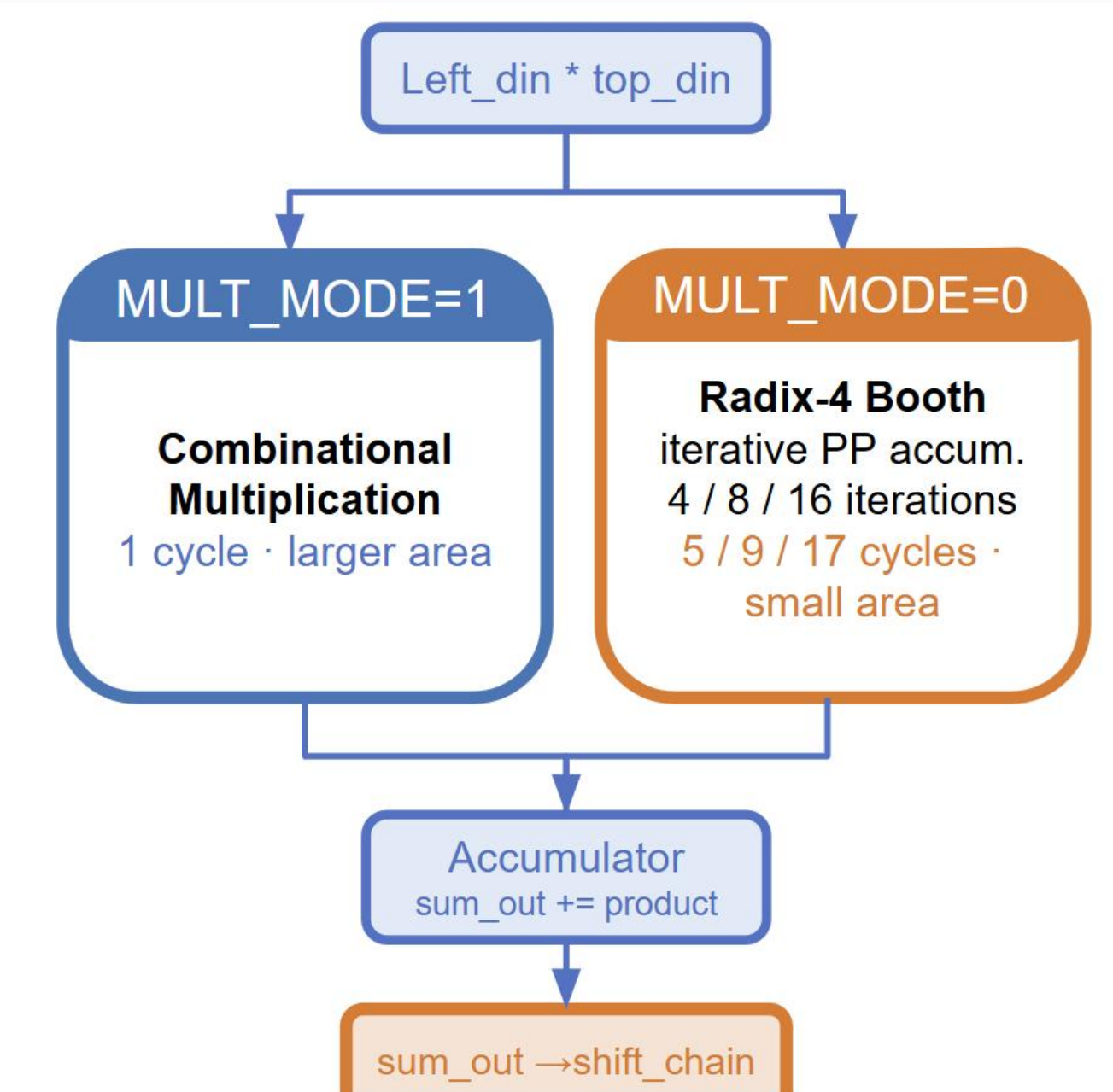
Tiling Strategy — Scale the matrix:

Partition a 32×32 GEMM into 16 tiles of 8×8. Rows are interleaved across 8 SRAM banks for conflict-free feeding. The tile is reused for multiple times to calculate the big matrix. The size of tile is parameterized. 8×8 tile is used to fit the resource on FPGA.

MAC Unit Design



The blue arrows in Fig.3 is SA calculation flow. Top and left data come from SRAM banks feeding into MAC unit. The orange arrows is accumulator shift chain used in data readout.



Tiling Computation: For each tile computation, 8 rows from the activation matrix and 8 cols from the weight matrix are taken and produce an 8 × 8 output tile. By iterating over different rows and columns, the Systolic Array is reused to generate all 16 tiles of the final 32 × 32 output matrix.

Support for different data types and different scenario need:

Precision: 4-bit to 32-bit dynamic support.

Two Modes: Combinational (High-speed) / Radix-4 Booth (Low-power & area).

Results for the Functional Design Part

FPGA Utilization

	Slice LUTs	Slice Registers	BRAM Tiles	DSPs
AI_ACC_TOP	18,176	13,479	8	0
SYSTOLIC_ARRAY	17,181 (94.5%)	12,160 (90.2%)	0 (0.0%)	0 (0.0%)
APB_SRAM_WRAPPER	895 (4.9%)	1,246 (9.2%)	8 (100%)	0 (0.0%)
SA2APB	75 (0.4%)	40 (0.3%)	0 (0.0%)	0 (0.0%)
ARRAY_CONTROLLER	25 (0.2%)	33 (0.3%)	0 (0.0%)	0 (0.0%)

ASIC Synthesis

Submodule	Area
AI_ACC_TOP (Total)	5,386,276 (100%)
APB_SRAM_WRAPPER	5,032,984 (93.4%)
SYSTOLIC_ARRAY (4x4)	346,212 (6.4%)
SA2APB	4,443 (0.08%)
ARRAY_CONTROLLER	2,637 (0.05%)

Standard Cell: 125,992 in total, including 53,894 combinational cells and 72,098 sequential/timing cells.

DRC:

Latch Count: 0 (Fully synchronous design)
Technology Macros: 0 (No SRAM macros)

Dynamic Power: ~59.4 mW @ 1.62V

Timing Constraint: VIOLATED at the target operational frequency of 10 MHz. The whole submodule's worst slack is -10,604 ns.

Verification methods and procedures

- **Unit-level:** Apply direct stimuli to verify blocks in the subsystem.
- **Subsystem-level:** Use a UVM-based environment with coverage-driven stimuli.
- **SoC-level:** Perform co-simulation between a direct testbench and C code tests.

Soc level Behavioral Simulation result

Test Mode	Input DWs (Act/Wei)	Run-time 1 4x4 SA	Run-time 2 8x8 SA
0	32bit/32bit	1218.3us	901.5 us
1	4bit/4bit	743.1us	584.7 us
2	8bit/8bit	743.1us	663.9 us
3	16bit/16bit	901.5us	743.1 us
4	4bit/32bit	1139.1us	822.3 us
5	8bit/32bit	1139.1us	822.3 us
6	8bit/4bit	743.1us	584.7 us
7	16bit/32bit	1139.1us	822.3 us
8	16bit/4bit	901.5us	743.1 us
9	16bit/8bit	901.5us	743.1 us

Code Coverage (Subsystem-level Verification)

Metrics	Coverage
Branch	100%
Condition	100%
Expression	100%
Line	100%
Toggle	100%
FSM	100%

Future Improvement

- Interrupt implementation
- Error detection mechanism
- Functional coverage verification
- 32-bit floating point support

*Note: Some coverage results have been excluded correctly to achieve above result.